

OOP : Assignment 1B

1. Explain the concept of programming paradigms and discuss their importance in Python.

Programming paradigms are different ways to write and organize code. Python supports multiple paradigms like procedural, object-oriented, and functional. This makes Python flexible and suitable for a wide range of applications.

2. Compare Object-Oriented Programming (OOP) with Procedural Programming, highlighting key differences.

- OOP uses classes and objects, and focuses on *data and behavior together*.
 - Procedural programming uses functions and focuses on *step-by-step instructions*.
 - OOP supports inheritance, encapsulation, and reusability, which procedural does not.
-

3. Describe the key advantages of using the OOP paradigm in Python with suitable examples.

- Reusability (via inheritance)
- Modularity (code is organized in classes)
- Encapsulation (data protection)

Example:

```
class Car:
```

```
    def drive(self):  
        print("Driving")
```

4. Discuss how Python implements data hiding in classes and its significance.

Python uses name conventions:

- `_var`: Protected (for internal use)
- `__var`: Private (name-mangled)

Significance: It helps prevent accidental access to sensitive data.

5. Explain the concept of classes in Python and demonstrate how to define a class with relevant examples.

A class is a blueprint for creating objects.

class Person:

```
def __init__(self, name):  
    self.name = name  
  
def greet(self):  
    print(f"Hello, {self.name}")
```

6. Describe the steps involved in creating and initializing an object from a class in Python.

1. Define a class.
2. Use `__init__()` to initialize data.
3. Create an object using class name.

```
p = Person("John")  
p.greet()
```

7. What is the role of the self-keyword in Python class methods? Explain with an example.

self refers to the current object. It allows access to attributes and methods.

```
class Dog:  
  
    def bark(self):  
        print("Woof")
```

8. Discuss the purpose and functionality of the `__init__()` method in Python classes with an example.

`__init__()` is the constructor. It initializes object attributes.

class Student:

```
def __init__(self, name):  
    self.name = name
```

9. Explain the concept of inheritance in Python. How does Python achieve inheritance in OOP?

Inheritance lets one class use features of another.

```
class Animal:  
    def sound(self): print("Sound")
```

```
class Dog(Animal): pass
```

Python uses class names in parentheses to achieve inheritance.

10. What is polymorphism in Python? Explain its role and provide suitable examples.

Polymorphism means using the same method name for different classes.

```
class Cat:  
    def speak(self): print("Meow")
```

```
class Dog:  
    def speak(self): print("Woof")
```

11. Define encapsulation in Python. How does it ensure data security in OOP?

Encapsulation means wrapping data and methods in one unit (class) and hiding internal details using private variables. It prevents direct access to sensitive data.

12. Explain abstraction in Python with an example. How does it improve code efficiency?

Abstraction hides complex implementation and shows only necessary parts.

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):
```

```
    @abstractmethod
```

```
    def area(self): pass
```

It makes code cleaner and easier to maintain.

13. Discuss how Python handles memory allocation for objects. What mechanisms are used for efficient memory management?

Python uses:

- Automatic memory management
- Garbage collection
- Reference counting

It frees memory when objects are no longer used.

14. What is a constructor in Python? Explain its purpose and demonstrate how to define one.

A constructor initializes new objects. In Python, it's `__init__()`.

```
class Book:
```

```
    def __init__(self, title):
```

```
        self.title = title
```

15. Describe the significance of the `__del__()` method in Python. When is it called, and how does it work?

`__del__()` is a destructor. It's called when an object is about to be deleted.

Code:

```
def __del__(self):
```

```
    print("Object deleted")
```

16. What is the role of the super() function in Python? How is it used in inheritance?

super() calls methods from the parent class.

class A:

```
def __init__(self):  
    print("A init")
```

class B(A):

```
def __init__(self):  
    super().__init__()  
    print("B init")
```

17. Explain the difference between class attributes and instance attributes in Python. Provide suitable examples.

- Class attribute: Shared across all instances.
- Instance attribute: Unique to each object.

class Dog:

```
species = "Mammal"    # class attribute  
  
def __init__(self, name):  
    self.name = name  # instance attribute
```

18. Discuss the purpose of static methods in Python. How are they different from instance methods?

Static methods don't use self and don't access object data. Use @staticmethod.

class Math:

```
@staticmethod  
def add(x, y):  
    return x + y
```

19. What is a friend function in Python? How is it implemented, and what are its uses?

Python doesn't support friend functions like C++. But you can access private data using name mangling (`_ClassName__var`). It's not recommended; use public methods instead.

```
class MyClass:
```

```
    def __init__(self):
```

```
        self.__secret = 42
```

```
def friend_func(obj):
```

```
    print(obj._MyClass__secret) # Accessing private member
```

```
obj = MyClass()
```

```
friend_func(obj)
```

20. Explain the concept of arrays of objects in Python. How can they be created and utilized effectively?

An array of objects is a list of instances.

```
class Student:
```

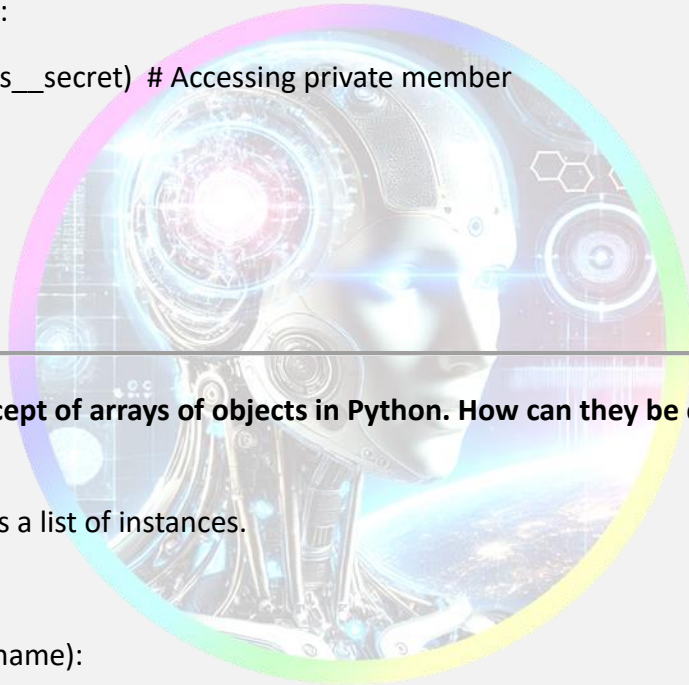
```
    def __init__(self, name):
```

```
        self.name = name
```

```
students = [Student("Alice"), Student("Bob")]
```

```
for s in students:
```

```
    print(s.name)
```



OOP : Assignment 2B

1. Explain the concept of polymorphism in Python with suitable examples. How does method overriding support polymorphism?

Polymorphism allows different classes to use the same interface (e.g., method name). Method overriding lets child classes redefine parent methods, enabling polymorphic behavior.

```
class Animal: def speak(self): return "Sound"
```

```
class Dog(Animal): def speak(self): return "Bark"
```

2. Discuss inheritance in Python. Describe different types of inheritance supported in Python with examples.

Inheritance lets a class reuse another class's methods/attributes. Types:

- Single
- Multiple
- Multilevel
- Hierarchical

```
class A: pass
```

```
class B(A): pass # Single
```

3. Describe the purpose and working of the super() function in Python. How does it simplify constructor calls in inheritance?

super() calls methods from the parent class, avoiding manual class name use. It helps manage constructors in complex inheritance.

```
class B(A): def __init__(self): super().__init__()
```

4. How does Python implement multiple inheritance? Explain with an example and discuss potential issues like the diamond problem.

Python uses Method Resolution Order (MRO) to resolve multiple inheritance.

Diamond problem occurs when multiple paths lead to the same ancestor. Python solves it using C3 linearization.

```
class A: pass; class B(A): pass; class C(A): pass; class D(B, C): pass
```

5. What is an abstract class in Python? Explain its significance in object-oriented programming and describe how to define one.

Abstract classes define interfaces and cannot be instantiated.

```
from abc import ABC
```

```
class Shape(ABC): pass
```

6. Explain the @abstractmethod decorator in Python. Why is it essential in creating abstract methods?

@abstractmethod forces subclasses to implement the method. Used in abstract base classes.

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC): @abstractmethod def draw(self): pass
```

7. Describe the role of the init() method in inheritance. How does Python manage constructor invocation in multi-level inheritance?

__init__() initializes objects. In multi-level inheritance, constructors must be explicitly or via super() called.

```
class A: def __init__(self): print("A")
```

```
class B(A): def __init__(self): super().__init__()
```

8. What is the purpose of the del() method in Python? Describe how destructors are invoked in inheritance scenarios.

`__del__()` is a destructor, called when an object is deleted. Inheritance follows method resolution for `__del__()`.

```
class A: def __del__(self): print("Deleted")
```

9. Explain the concepts of composition and aggregation in Python with examples. How do they differ from each other?

- Composition: strong ownership; part dies with whole.
- Aggregation: weak association; part lives independently.

```
class Engine: pass
```

```
class Car: def __init__(self): self.engine = Engine() # Composition
```

10. How can a child class in Python call a method from its parent class? Explain different approaches with examples.

Use `super().method()` or `ParentClass.method(self)`.

```
super().display()
```

```
# or
```

```
Parent.display(self)
```

11. Discuss the purpose of the `isinstance()` function in Python. How does it help in type checking during inheritance?

Answer:

- `isinstance()` checks if an object is an instance of a class or its subclasses.
- It's useful for ensuring type compatibility, especially in inheritance hierarchies.

```
isinstance(obj, ParentClass)
```

12. Explain type conversion from basic data types to user-defined types in Python. Provide examples to demonstrate the process.

Answer:

- You can convert a basic data type to a user-defined type by passing it as an argument to a class constructor.

class Age:

```
def __init__(self, years):  
    self.years = int(years)
```

```
a = Age("25")
```

13. How can you convert a user-defined data type back into a basic data type in Python? Illustrate this with an example.

Answer:

- Implement special methods like `__int__()`, `__str__()`, or `__float__()` to define how an object converts back.

class Age:

```
def __init__(self, years): self.years = years  
def __int__(self): return self.years
```

```
a = Age(30)
```

```
print(int(a)) # Output: 30
```

14. What is a metaclass in Python? Explain how metaclasses differ from regular classes and their significance in object creation.

Answer:

- A metaclass is a class of classes, controlling class creation (not object creation).
- While classes create objects, metaclasses create classes.

```
class Meta(type): pass

class MyClass(metaclass=Meta): pass
```

15. Discuss the role of unit testing in Python development. Why is unit testing important, and how can you write a simple unit test using the unittest module?

Answer:

- Unit testing checks if individual components work correctly.
- Ensures reliability and simplifies debugging.

```
import unittest

class TestMath(unittest.TestCase):

    def test_add(self):

        self.assertEqual(2 + 2, 4)
```

16. Describe the purpose and usage of the assert statement in Python. How does it differ from raising exceptions directly?

Answer:

- assert tests conditions during development; raises AssertionError if false.
- Simpler than manual raise, but can be disabled in optimized mode.

```
assert x > 0, "x must be positive"
```

17. Explain the try-except block in Python. How can it be effectively used to handle different types of exceptions? Provide examples.

Answer:

- try-except is used to catch and handle runtime errors.
- You can handle multiple error types separately.

```
try:  
    a = 1 / 0  
except ZeroDivisionError:  
    print("Division by zero")  
except Exception as e:  
    print(e)
```

18. How can you define and raise custom exceptions in Python? Provide a practical example demonstrating custom exception handling.

Answer:

- Create a custom class inheriting from Exception.
- Use raise to trigger the custom error.

```
class MyError(Exception): pass  
raise MyError("This is a custom error")
```

19. Discuss the concept of hierarchical inheritance in Python. How does Python manage constructor invocation in such scenarios?

Answer:

- One parent class is inherited by multiple child classes.
- Each child class must call the parent constructor using `super()` to initialize inherited attributes.

```
class A: def __init__(self): print("A init")  
class B(A): def __init__(self): super().__init__()  
class C(A): def __init__(self): super().__init__()
```

20. Explain the `__str__()` method in Python. How does it enhance the readability of objects, and why is it important in class design?

Answer:

- `__str__()` returns a human-readable string representation of an object.
- Used by `print()` and helps in debugging/logging.

class Person:

```
def __init__(self, name): self.name = name
```

```
def __str__(self): return f"Person: {self.name}"
```

